Enhancing xv6

CPU Time Accounting and Priority-Based Scheduling

Final Project Report

Course: CSE323 Operating Systems

Submitted To: Farhan Tanvir Khan

Submitted By

Submitted By	Student ID
Akib Hassan Rifat	2312379042
Tauhid Ara Himu	2312187042

Submission Date: May 6, 2026

Abstract

This project extends xv6-public with two process-management features: per-process CPU time accounting and priority-based scheduling. The original xv6 scheduler is intentionally simple and does not report how much CPU time each process has used. We added kernel-level accounting fields, new system calls, and user programs to make that information visible. We also modified the scheduler so that runnable processes with lower numeric priority values are selected before lower-importance processes. The final implementation compiled successfully, the custom demonstration programs produced the expected results, and the xv6 usertests suite completed with ALL TESTS PASSED.

Introduction

Background

xv6 is a compact Unix-like teaching operating system used to study core OS ideas such as processes, traps, system calls, context switching, and scheduling. Its simplicity is useful for learning, but it also means that several practical operating-system features are missing. In the stock xv6-public code, user programs cannot ask the kernel how much CPU time a process has consumed, and the scheduler treats runnable processes with equal weight.

This project addresses those two limitations. The CPU accounting feature gives user programs a controlled view of process CPU usage. The priority scheduler adds a basic scheduling policy where important processes can be selected before less important ones.

Objectives

- Track CPU usage for each process using xv6 timer ticks.
- Expose process details through a new getprocinfo system call.
- Create user programs to display and validate CPU accounting.
- Add fixed-priority scheduling with lower values representing higher priority.
- Provide setpriority and getpriority system calls.
- Verify the changes using custom tests and xv6 usertests.

Literature Review

Process scheduling and CPU accounting are central responsibilities of an operating system. A scheduler decides which runnable process receives the CPU, while accounting records how system resources are consumed. Production systems provide similar information through tools and interfaces such as top, ps, /proc, getrusage(), and time-related kernel accounting interfaces.

Round-robin scheduling, the default approach in xv6, is simple and fair for equal-priority work. However, it does not support policy decisions where some tasks should run before others. Priority scheduling is a common extension: each process is assigned a priority, and the scheduler favors the highest-priority runnable process. A limitation of fixed priority scheduling is starvation, which can be reduced in future work by priority aging.

The xv6 book describes how process states and context switching interact. Based on that design, the scheduler is the natural place for CPU accounting because it marks the exact boundary where a process begins and stops using the CPU.

Linux documentation is also relevant because it separates process time from elapsed time and exposes CPU usage through resource-usage interfaces. The priority part of this project is related to the same broader idea behind scheduler parameter interfaces: the kernel maintains policy information and uses it when choosing runnable work.

Methodology

Development Setup

Item	Description
Codebase	xv6-public x86
Environment	Ubuntu/WSL with QEMU
Build commands	make clean, make, make qemu-nox

CPU Time Accounting

Three fields were added to struct proc. cpu_ticks_total stores accumulated CPU usage. cpu_ticks_start stores the tick value at the moment the process is scheduled in. The priority field is used by the second feature and is also shown by the process information tool.

```
int priority;
uint cpu_ticks_total;
uint cpu_ticks_start;
```

Every process is initialized in allocproc(), so the accounting fields start from a known value for init, shell, and all forked processes.

```
p->priority = 5;
p->cpu_ticks_total = 0;
p->cpu_ticks_start = 0;
```

The accounting logic is placed around the scheduler context switch. When the process starts running, the scheduler records the current tick value. When control returns to the scheduler, the elapsed ticks are added to the total.

```
p->state = RUNNING;
p->cpu_ticks_start = ticks;

swtch(&(c->scheduler), p->context);

p->cpu_ticks_total += ticks - p->cpu_ticks_start;
p->cpu_ticks_start = 0;
```

System Calls and User Interface

A small shared structure named procinfo was added so the kernel can safely return process information to user space. The getprocinfo(pid, buf) system call fills this structure with PID, process name, state, priority, and CPU tick count. The user pointer is checked with argptr before the kernel writes to it.

System call	Purpose
getprocinfo(pid, buf)	Returns process name, state, priority, and CPU ticks.
setpriority(pid, priority)	Changes the priority of a process.
getpriority(pid)	Returns the priority of a process.

Priority Scheduling

The scheduler was changed from pure round-robin selection to fixed-priority selection. It first scans the process table to find the lowest priority value among RUNNABLE processes. It then runs processes that match that best priority. This preserves round-robin behavior among processes with the same priority.

```
best_priority = 0x7fffffff;
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
  if(p->state != RUNNABLE)
    continue;
  if(p->priority < best_priority)
    best_priority = p->priority;
}
```

Files Modified

File	Role in the implementation
proc.h / proc.c	Added fields, scheduler accounting, priority selection, and kernel helper functions.
procinfo.h	Defined the user-visible process information structure.
sysproc.c	Implemented syscall handlers and argument validation.
syscall.h / syscall.c	Assigned syscall numbers and connected the syscall table.
user.h / usys.S	Added user-space prototypes and syscall stubs.
Makefile	Included cpuusage, accounttest, and prioritytest in the xv6 file system.

Results and Findings

The project was tested by building xv6, booting it in QEMU, running the new demonstration programs, and finally running the official xv6 usertests suite.

CPU Accounting Result

```
$ accounttest
busy child finished work: 1283106752
busy pid 5 used 15 ticks
sleeping pid 6 used 0 ticks
```

The result shows the main idea clearly: a CPU-bound process accumulates CPU ticks, while a sleeping process does not consume CPU time.

Process Information Output

```
$ cpuusage 1
PID     STATE       PRIO     TICKS     NAME
1       sleep       5        3         init
2       sleep       5        0         sh
3       running     5        1         cpuusage
```

Priority Scheduling Result

```
$ prioritytest
pid 8 priority set to 10
pid 9 priority set to 5
pid 10 priority set to 1
child 2 pid 10 priority 1 ticks 21 done
child 1 pid 9 priority 5 ticks 23 done
child 0 pid 8 priority 10 ticks 22 done
```

The process with priority 1 finished before priority 5 and priority 10, confirming that lower numeric priority values are preferred by the modified scheduler.

Regression Test

```
$ usertests
ALL TESTS PASSED
```

Findings

• CPU accounting correctly separates CPU execution time from sleeping time.

• The new getprocinfo system call exposes useful process information without exposing kernel-only struct proc fields.

• Priority scheduling works for CPU-bound test processes.

• The modified kernel remains stable under xv6 usertests.

Conclusion and Recommendations

The project successfully adds two useful process-management features to xv6-public. CPU time accounting makes process behavior measurable, while priority scheduling introduces a simple but practical scheduling policy. The implementation required changes in the process structure, scheduler, syscall layer, and user programs, which made the project a strong exercise in kernel-level process management.

The tests confirm that the implementation behaves as expected: CPU-bound processes gain ticks, sleeping processes do not, and higher-priority processes are scheduled first. The successful usertests result also indicates that the new functionality did not break the base xv6 system.

Recommendations

- Add priority aging to reduce starvation risk for low-priority processes.
- Use wider counters if very long-running accounting is required.
- Extend cpuusage with filtering options for PID or process name.
- Record additional metrics such as creation time, sleep time, and turnaround time.

References

1. MIT PDOS. xv6-public source code. Available at: https://github.com/mit-pdos/xv6-public

2. Russ Cox, Frans Kaashoek, and Robert Morris. xv6: a simple, Unix-like teaching operating system. MIT PDOS. Available at: https://ocw.mit.edu/courses/6-828-operating-system-engineering-fall-2012/3def8fcd397933ebb846fb479bdcf556_MIT6_828F12_xv6-book-rev7.pdf

3. Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces, Version 1.10. Available at: https://pages.cs.wisc.edu/~remzi/OSTEP/

4. Michael Kerrisk. getrusage(2) - Linux manual page. Linux man-pages project. Available at: https://man7.org/linux/man-pages/man2/getrusage.2.html

5. Michael Kerrisk. time(7) - Linux manual page. Linux man-pages project. Available at: https://man7.org/linux/man-pages/man7/time.7.html

6. Michael Kerrisk. sched_setscheduler(2) - Linux manual page. Linux man-pages project. Available at: https://man7.org/linux/man-pages/man2/sched_setscheduler.2.html

7. OSTEP Projects. Available at: https://github.com/remzi-arpacidusseau/ostep-projects